HCMC University of Technology, VNU



Software Architecture

Chapter 5. Architecture Styles (cont.)

Lecturer: Thai-Minh Truong, PhD.

Email: thaiminh@hcmut.edu.vn

CHAPTER 5. Advanced Architecture Styles



5.8. Event-Driven Architecture Style

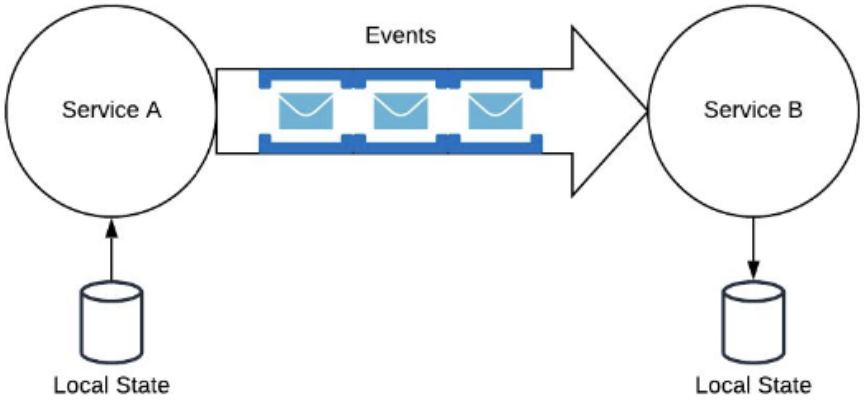
5.9. Space-Based Architecture Style (Self-study)

5.8. Event-Driven Architecture Style



Introduction

- It is a popular distributed asynchronous architecture styles and can be used for scalable and high-performance applications.
- It is also highly adaptable and can be used for small applications and as well as large, complex ones.
- It can be used as a standalone architecture style or embedded within other architecture styles.
- Decoupled event processing components that async receives and process events.



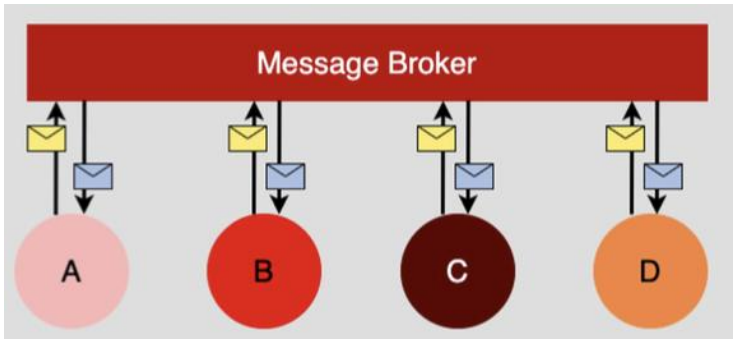
5.8. Event-Driven Architecture Style



Topology



The mediator topology: *In case you require control over the workflow of an event process*

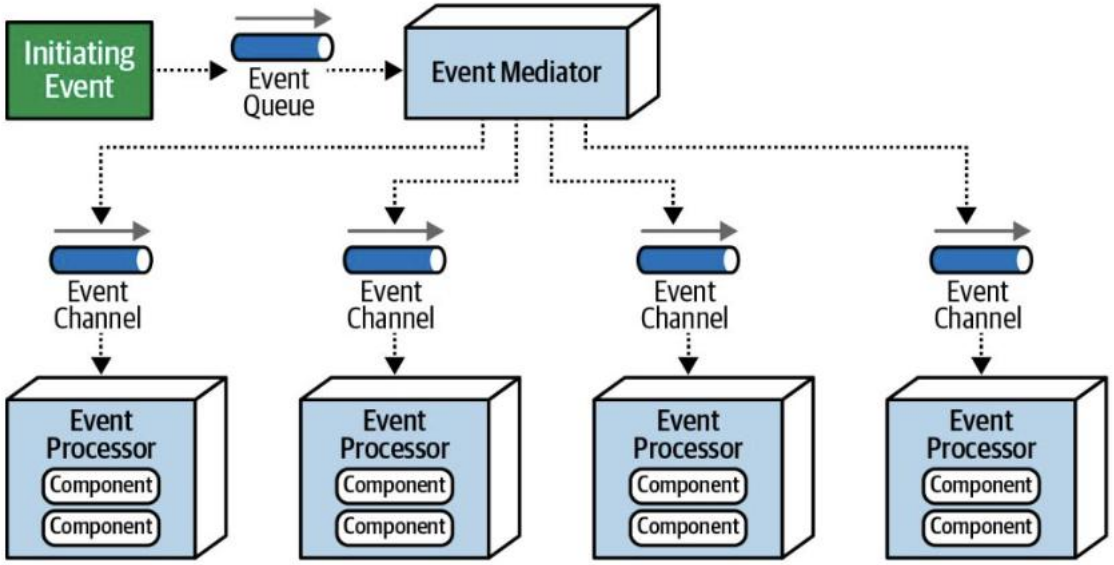


The broker topology: *In case you require a high. degree of responsiveness and dynamic control over the processing of an event.*

5.8. Event-Driven Architecture Style



Mediator Topology

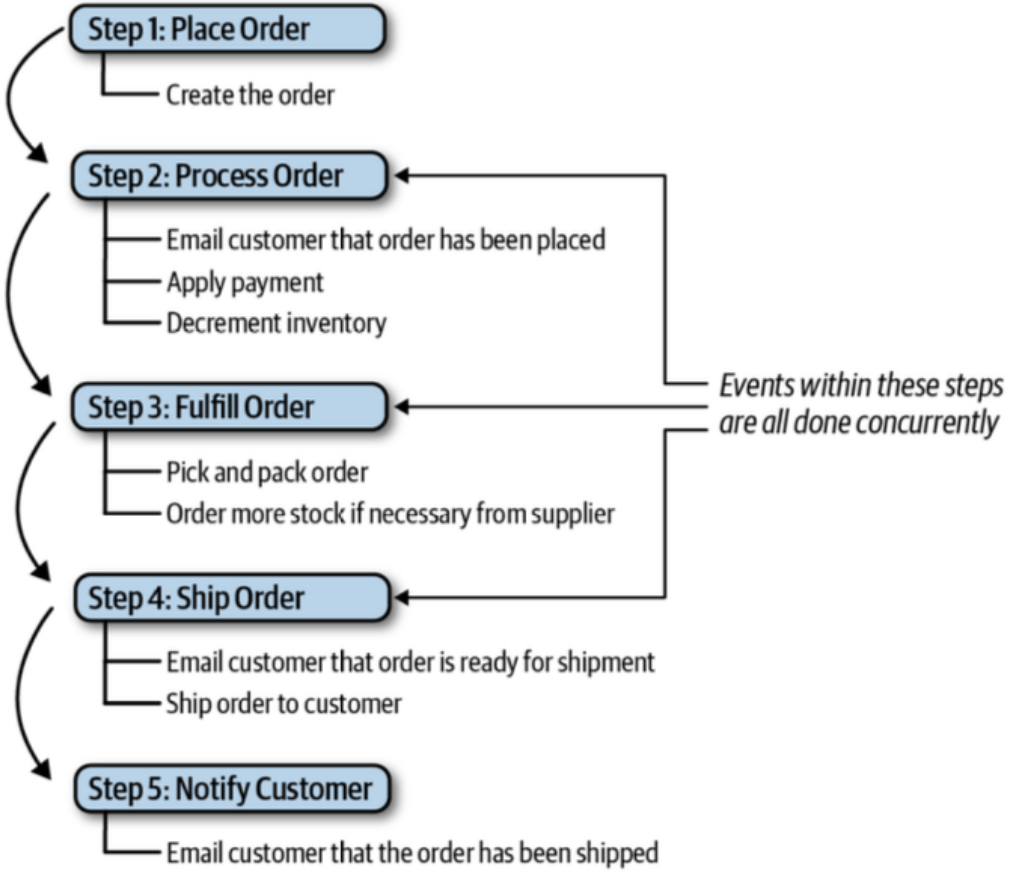


- Center is an event mediator, It includes event, event queue, event mediator, event channels and event processor
- Do not advertise what it did to the rest of the system like broker topology

5.8. Event-Driven Architecture Style

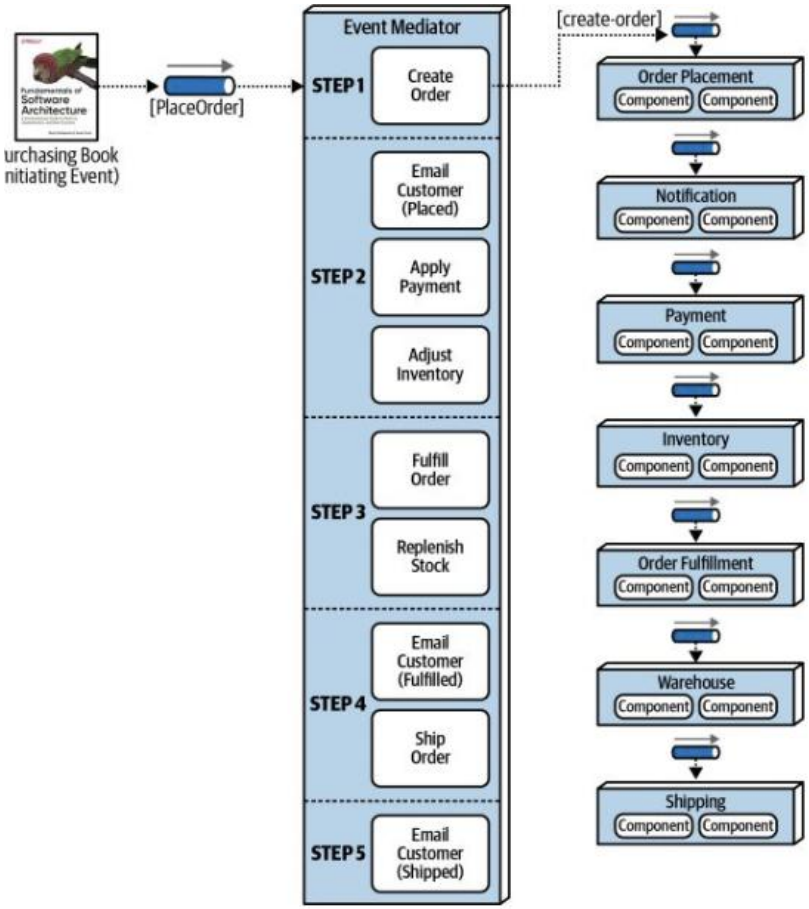


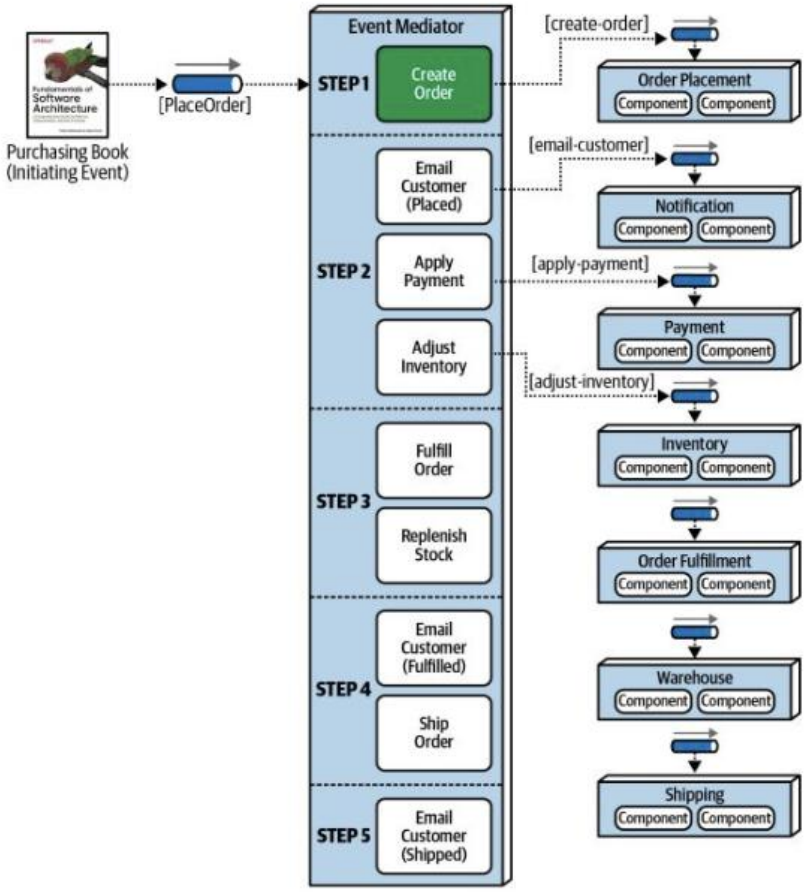
Mediator Topology Example



5.8. Event-Driven Architecture Style

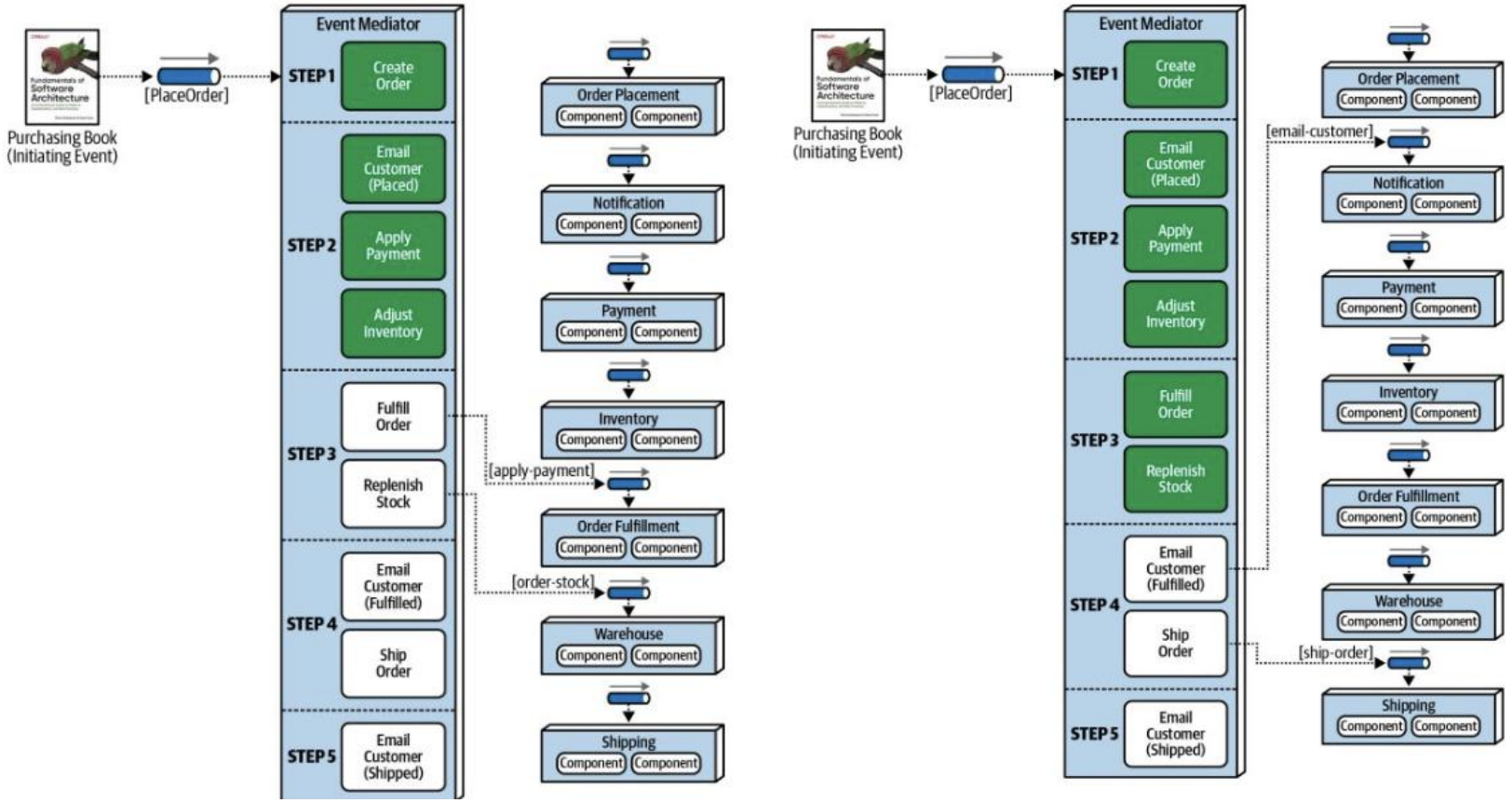
Mediator Topology Example





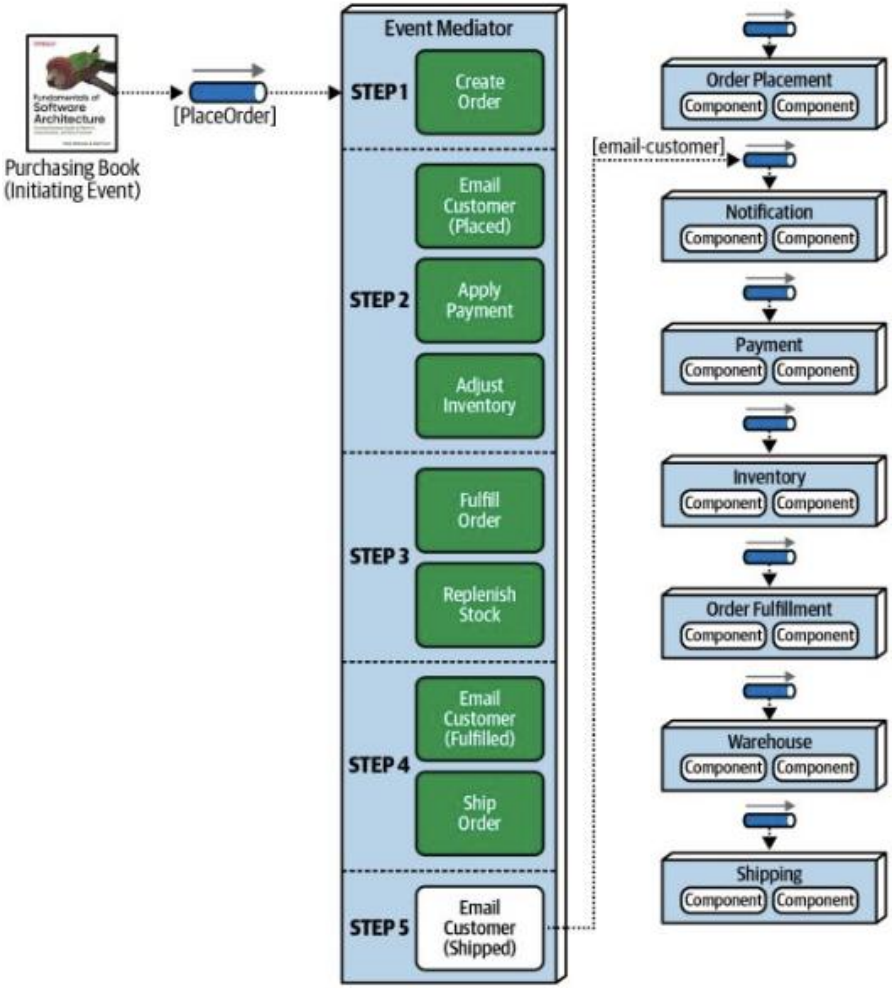
5.8. Event-Driven Architecture Style

Mediator Topology Example



5.8. Event-Driven Architecture Style

Mediator Topology Example



5.8. Event-Driven Architecture Style



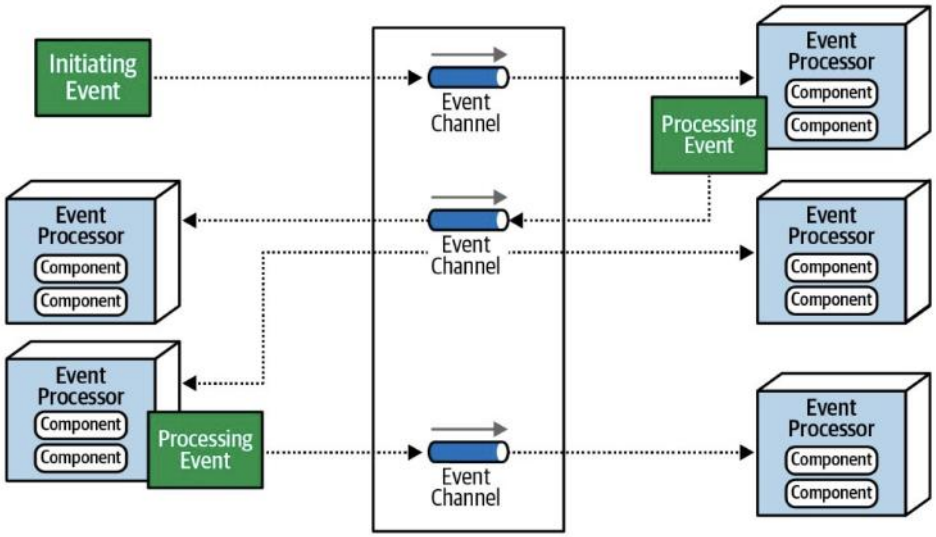
Mediator Topology

Advantages	Disadvantages
Workflow control	More coupling of event processors
Error handling	Lower scalability
Recoverability	Lower performance
Restart capabilities	Lower fault tolerance
Better data consistency	Modeling complex workflows

5.8. Event-Driven Architecture Style



Broker Topology

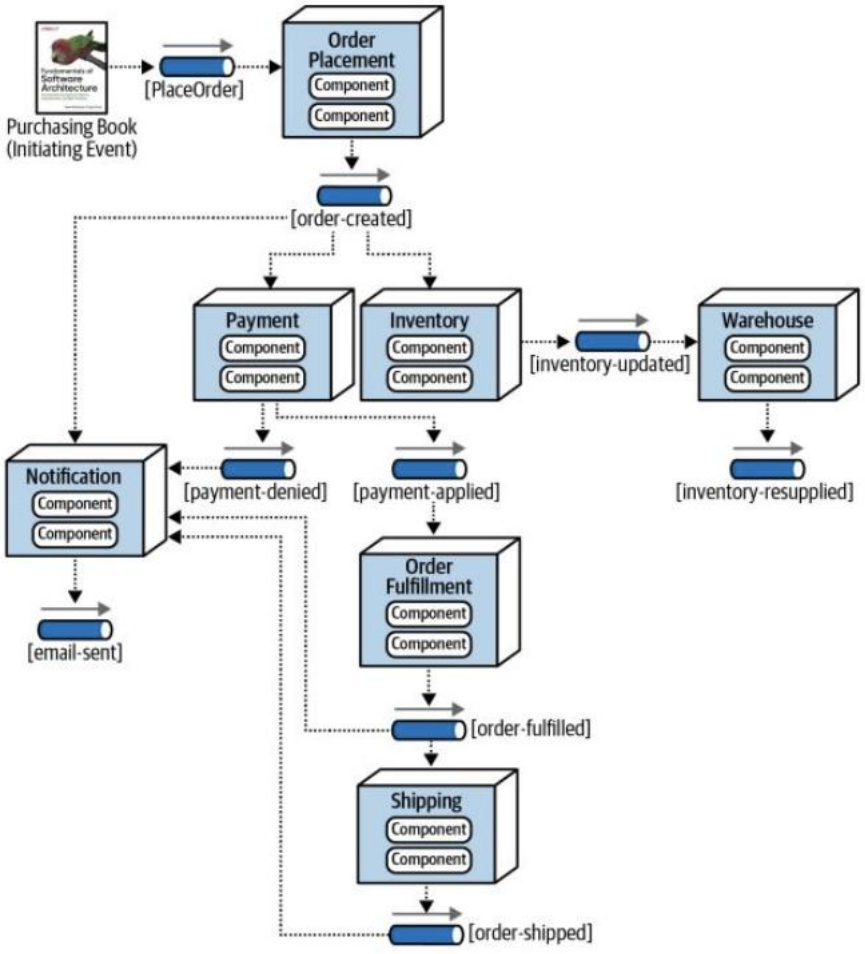


- 4 primary components: a initiating event, the event broker, an event processor, and processing event
- No center event mediator, the event is distributed across the event processor by broadcasting fashion through a broker like RabbitMQ

5.8. Event-Driven Architecture Style



Broker Topology Example



- All of the event processors are highly decoupled and independent of each other.
- Non control overall workflow. Lead to handling Error is a big challenge
- Performance is very well and easy to scale.
- No monitor and manage by mediator so awareness of component crash

5.8. Event-Driven Architecture Style



Broker Topology

Advantages	Disadvantages
Highly decoupled event processors	Workflow control
High scalability	Error handling
High responsiveness	Recoverability
High performance	Restart capabilities
High fault tolerance	Data inconsistency

5.8. Event-Driven Architecture Style



Broker Topology

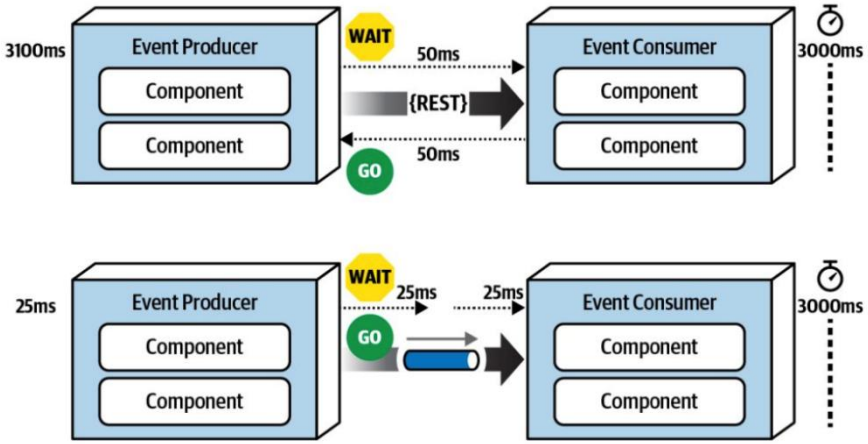
In-class activity: Compare RabbitMQ and Apache Kafka

5.8. Event-Driven Architecture Style



Asynchronous Capabilities

- Event-driven relies solely on asynchronous communication for both fire-and-forget processing (no response required) as well as request/reply processing (response required from the event consumer)



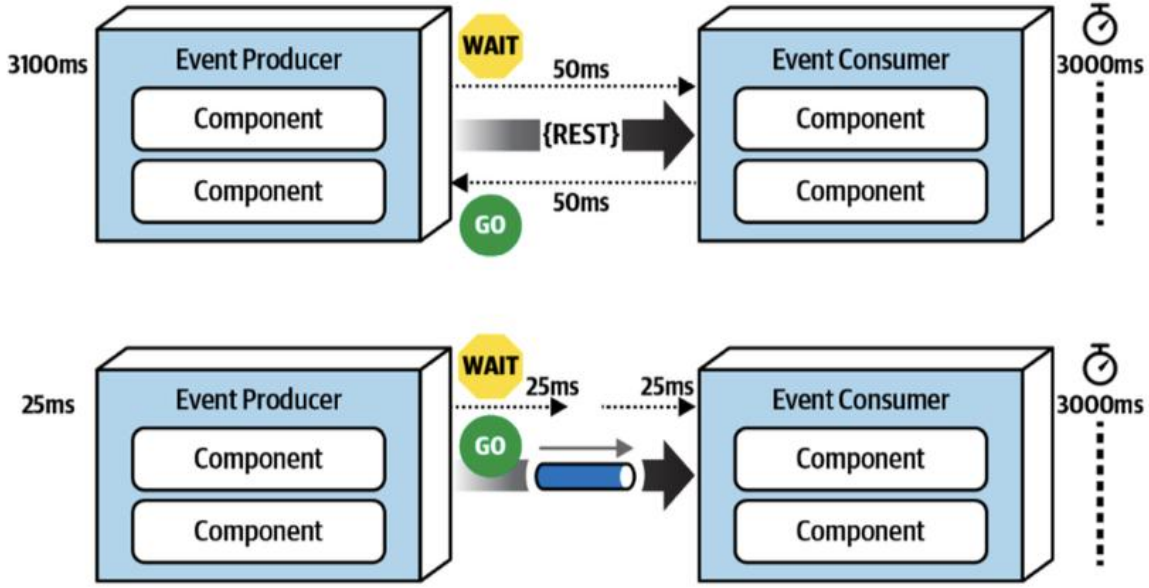
=> Responsiveness

5.8. Event-Driven Architecture Style



Asynchronous Capabilities



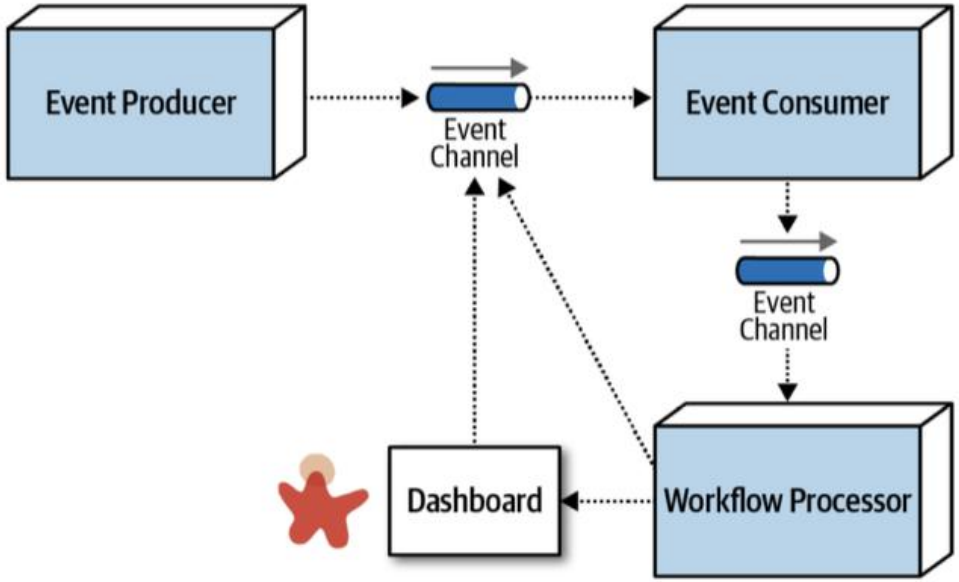


5.8. Event-Driven Architecture Style



Error Handling

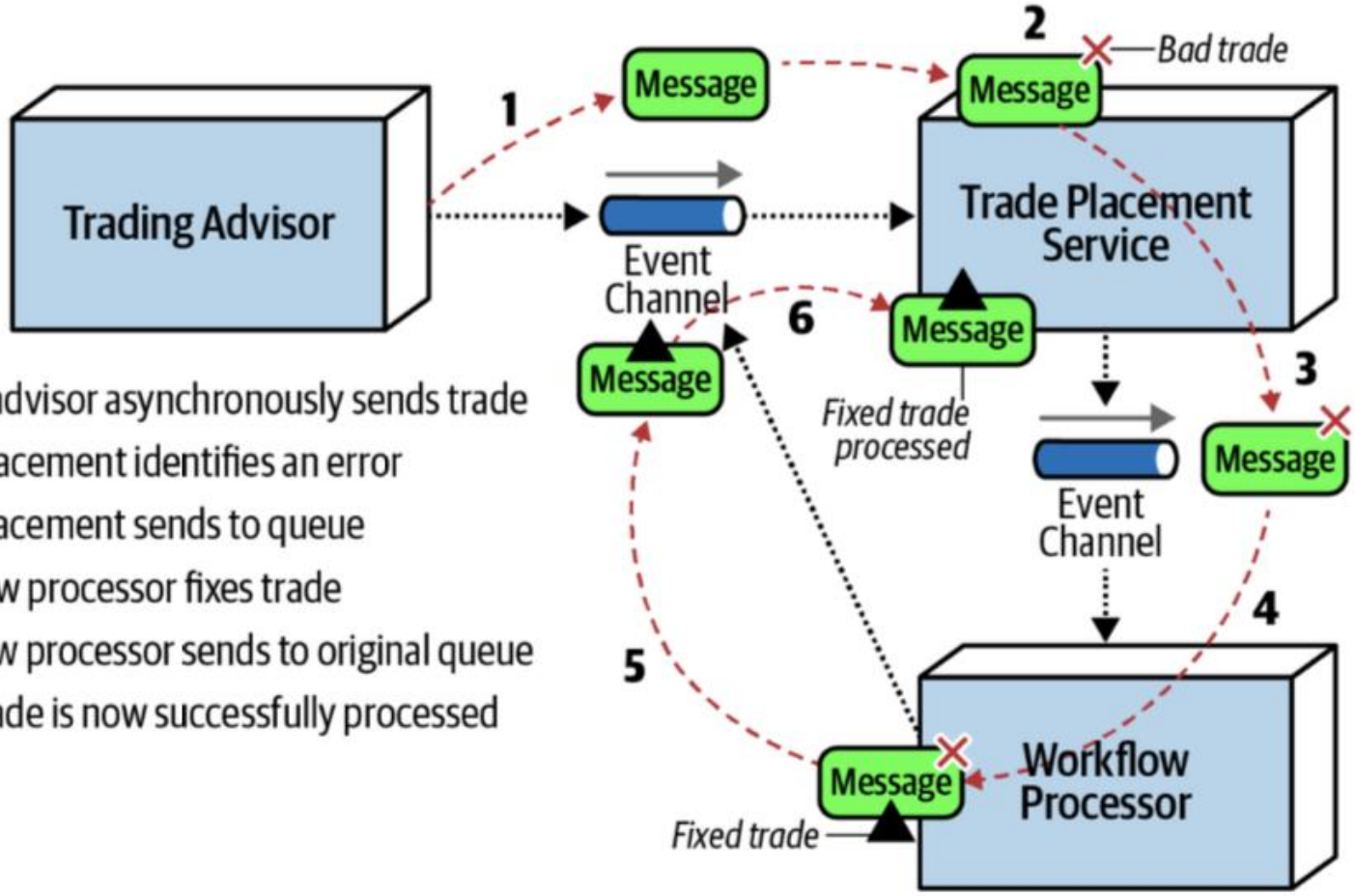
- The **workflow event pattern** of reactive architecture is one way of addressing the issues associated with error handling in an asynchronous workflow through the use of a **workflow processor**.



5.8. Event-Driven Architecture Style



Error Handling Example



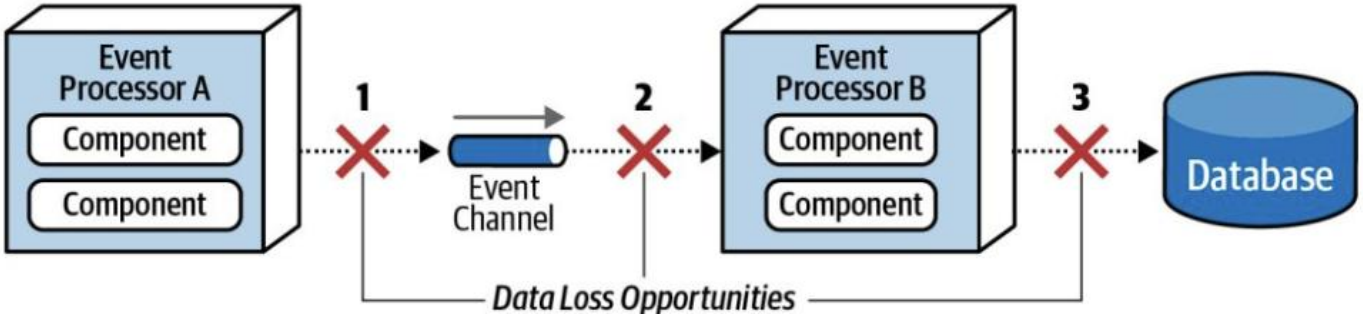
1. Trading advisor asynchronously sends trade
2. Trade placement identifies an error
3. Trade placement sends to queue
4. Workflow processor fixes trade
5. Workflow processor sends to original queue
6. Fixed trade is now successfully processed

5.8. Event-Driven Architecture Style



Prevent Data Loss

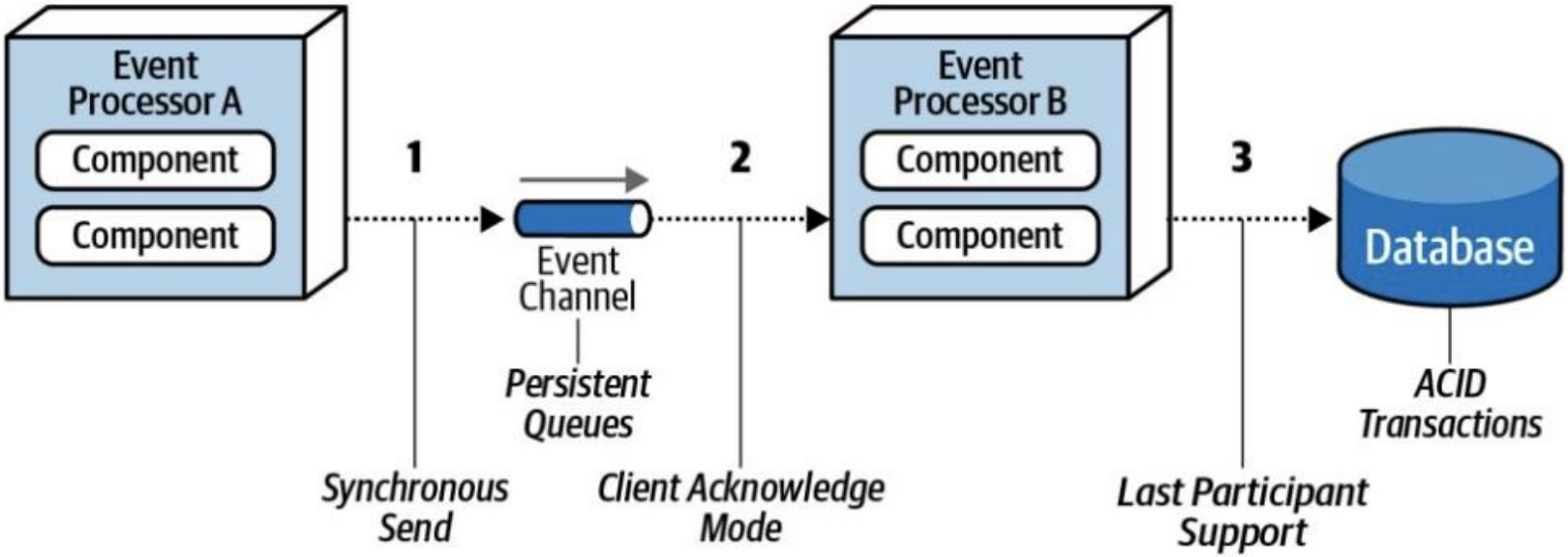
- Data loss is always a primary concern when dealing with asynchronous communications.
- Three areas of data loss can occur.



5.8. Event-Driven Architecture Style



Prevent Data Loss

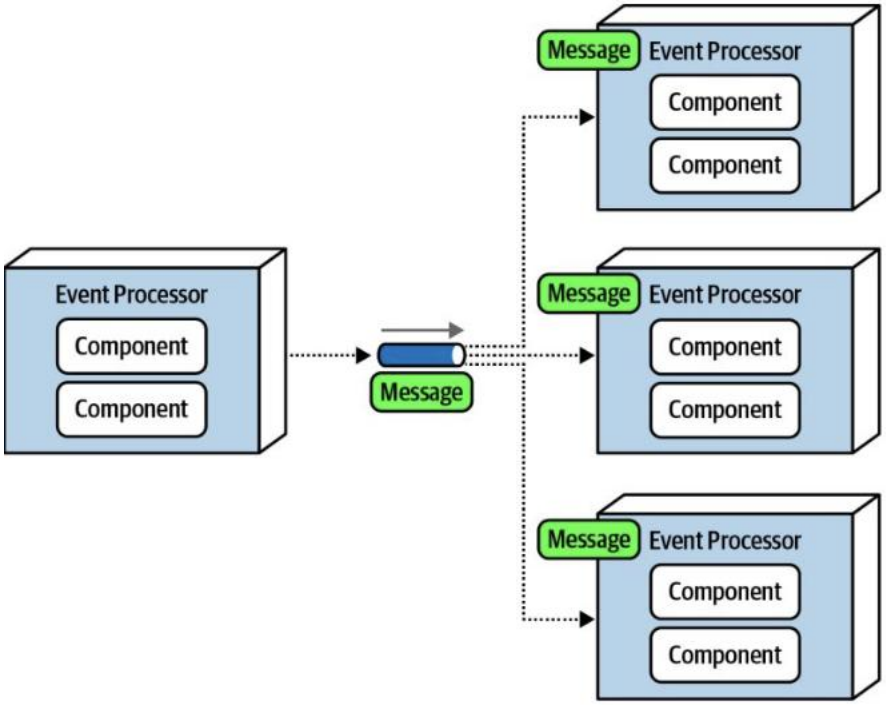


5.8. Event-Driven Architecture Style



Broadcast Capabilities

- The capability to broadcast events without knowledge of who (if anyone) is receiving the message and what they do with it

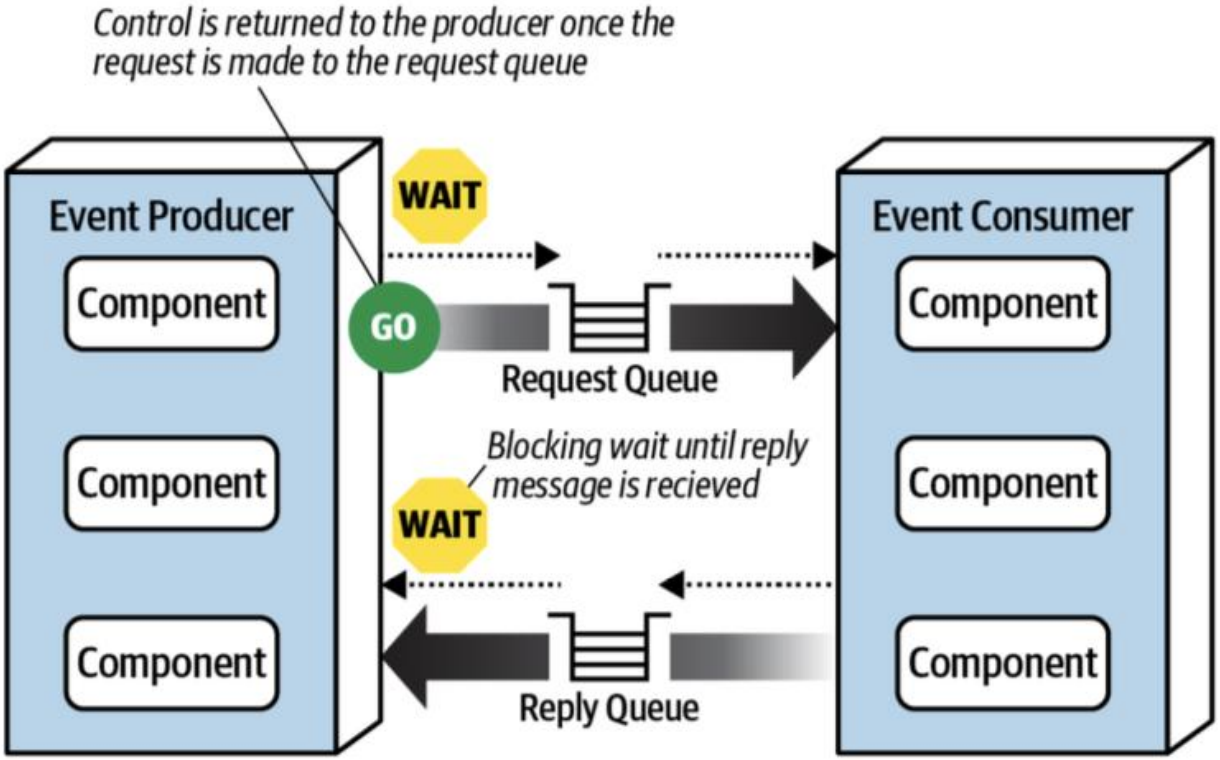


5.8. Event-Driven Architecture Style



Request - Reply

- In event-driven architecture, synchronous communication is accomplished through **request-reply messaging**

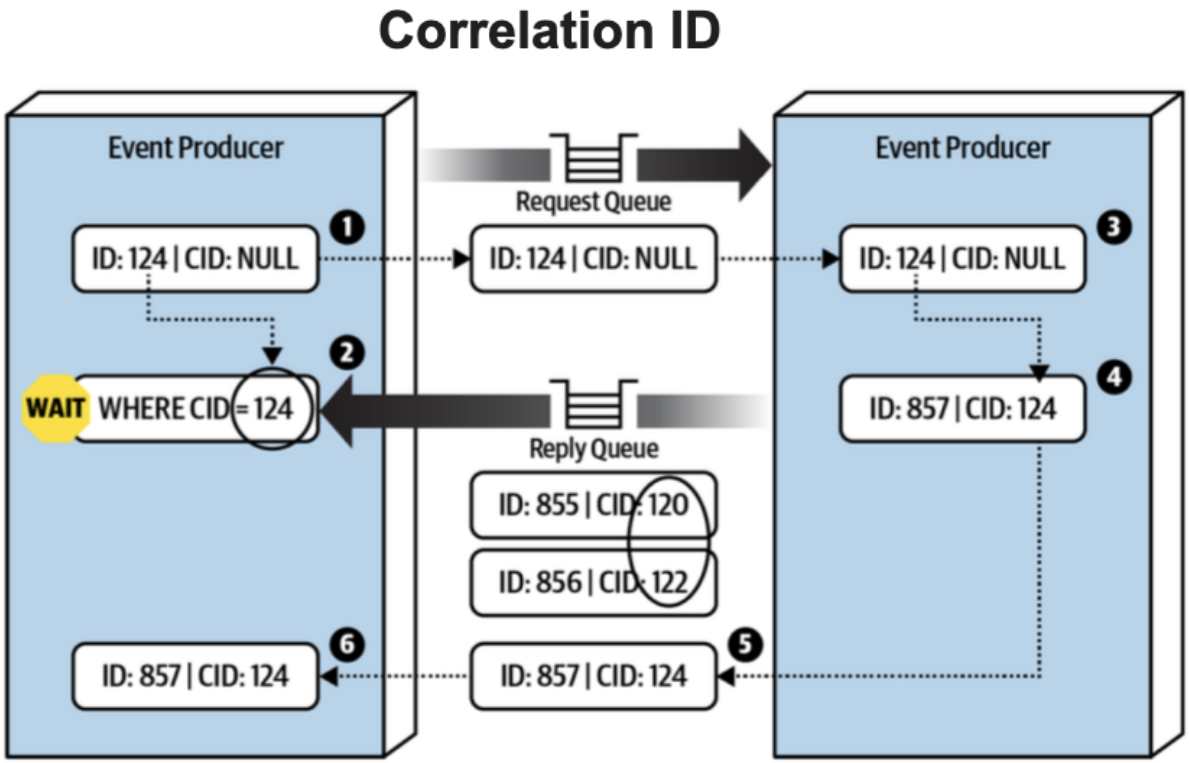


5.8. Event-Driven Architecture Style



Request - Reply

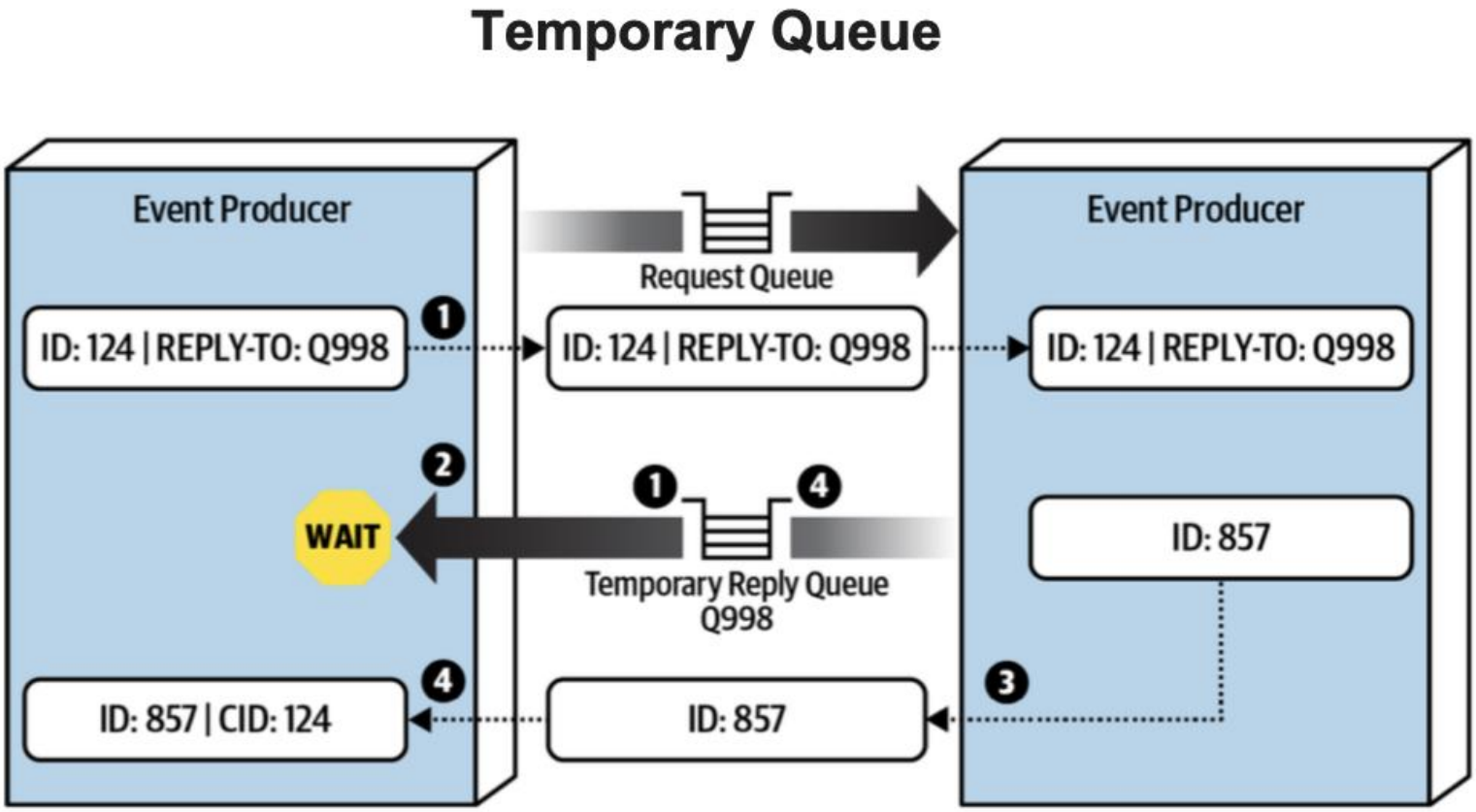
- There are **two primary techniques** for implementing request-reply messaging.



5.8. Event-Driven Architecture Style



Request - Reply



5.8. Event-Driven Architecture Style



Request vs Event-based

Advantages over request-based	Trade-offs
Better response to dynamic user content	Only supports eventual consistency
Better scalability and elasticity	Less control over processing flow
Better agility and change management	Less certainty over outcome of event flow
Better adaptability and extensibility	Difficult to test and debug
Better responsiveness and performance	
Better real-time decision making	
Better reaction to situational awareness	

5.8. Event-Driven Architecture Style



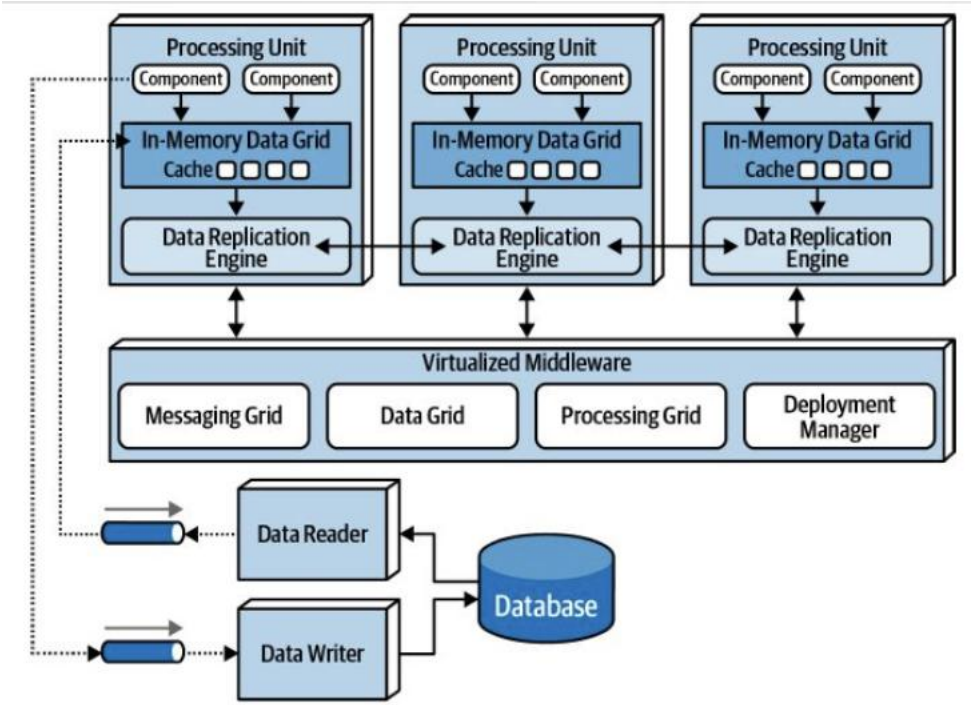
Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1 to many
Deployability	★★★
Elasticity	★★★
Evolutionary	★★★★★
Fault tolerance	★★★★★
Modularity	★★★★
Overall cost	★★★
Performance	★★★★★
Reliability	★★★
Scalability	★★★★★
Simplicity	★
Testability	★★

5.9. Space-Based Architecture Style



Topology

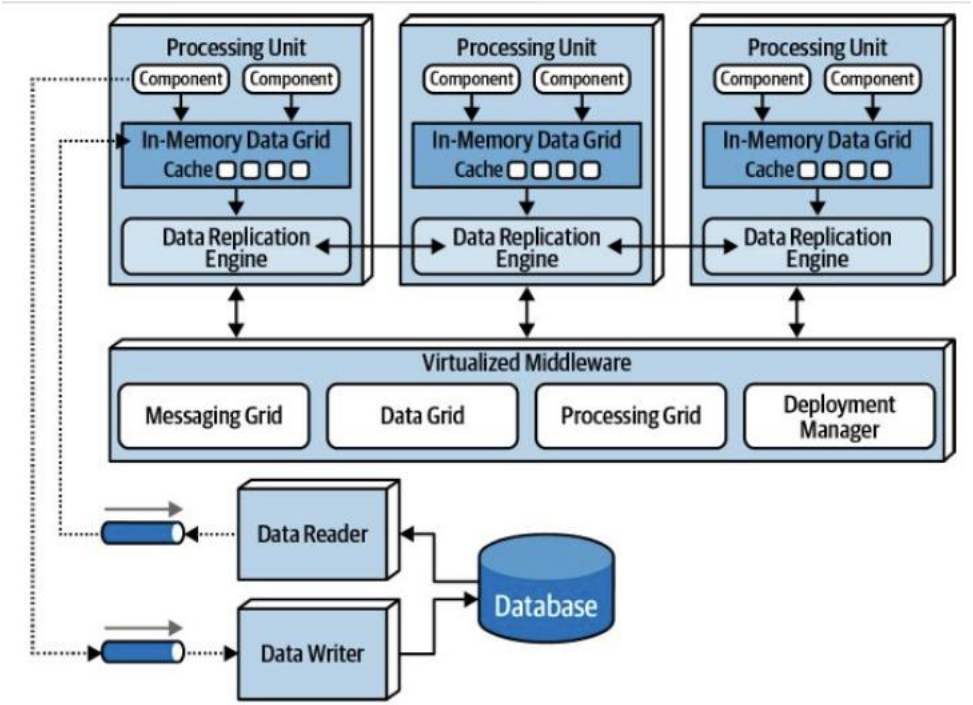


- The space-based pattern (the cloud architecture pattern) minimizes the factors that limit application scaling. (This pattern gets its name from the concept of tuple space - the idea of distributed shared memory.)
- Multiple parallel processors communicating through shared memory
- Processing units start up and shut down dynamically
- There is no central database

5.9. Space-Based Architecture Style



Topology

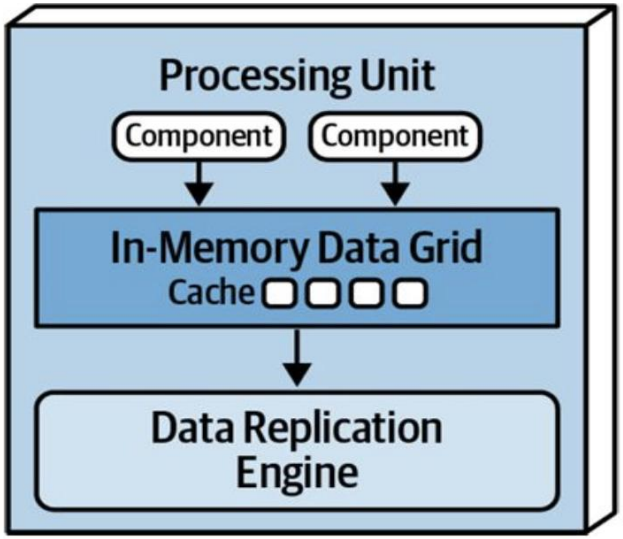


- There are several architecture components that make up a space-based architecture:
  - **A processing unit** containing the application code
  - The **virtualized middleware** used to manage and coordinate the processing units
  - **Data pumps** to asynchronously send updated data to the database
  - **Data writers** that perform the updates from the data pumps
  - **Data readers** that read database data and deliver it to processing units upon startup.

5.9. Space-Based Architecture Style



Processing unit



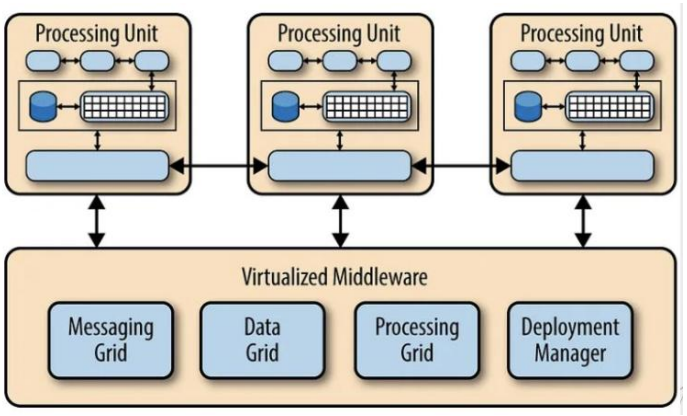
- The processing unit contains the the **application unit**
- It usually includes web-based components as well as backend business logic.
- In addition to the application logic, the processing unit also contains an in-memory data grid and replication engine usually implemented through such products as *Hazelcast, Apache Ignite,* and *Oracle Coherence*.

- Smaller web-app (deploy into single processing unit)
- Larger app may split the application functionality into multiple processing units (based on functional requirement)

5.9. Space-Based Architecture Style



Virtualized middleware

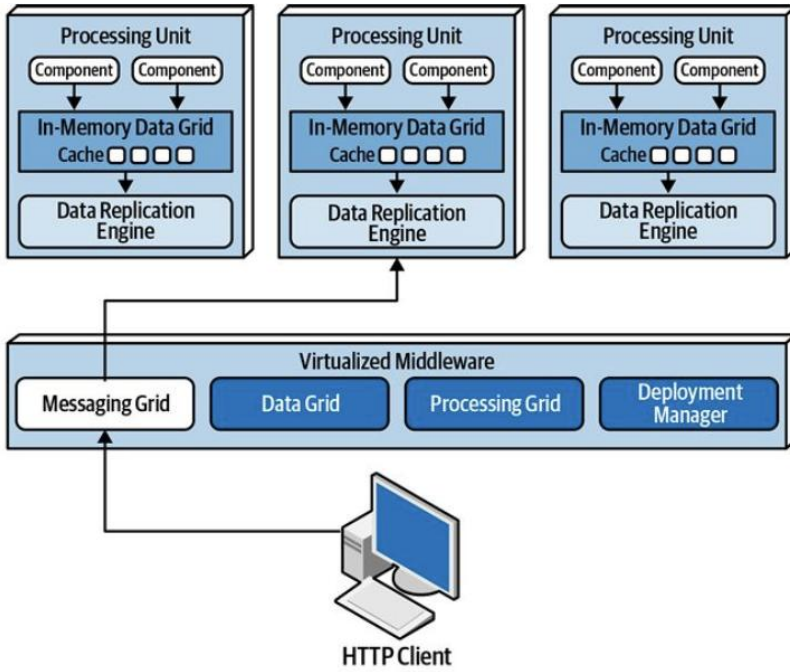


- The virtualized middleware **handles the infrastructure concerns** within the architecture that control various aspects of **data synchronization and request handling**
- The components that make up the virtualized middleware include a **messaging grid, data grid, processing grid, and deployment manager.**

5.9. Space-Based Architecture Style



Massaging grid

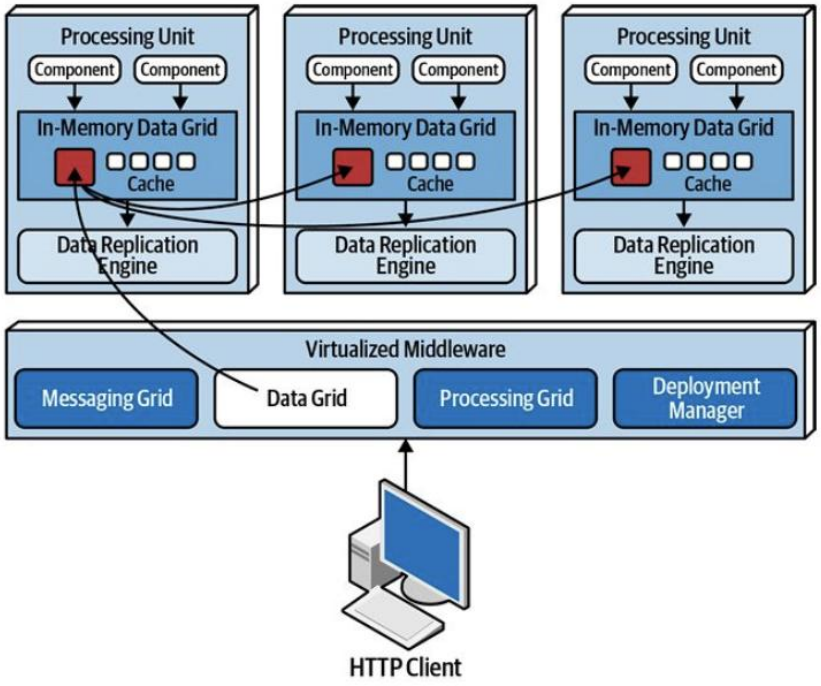


- Manages **input request** and **session rate**
- When a request comes into the virtualized middleware, the messaging grid component determines which active processing components are available to receive the request and forwards the request to one of those processing units.
- Complexity: simple RR algorithm to more complex next-available algorithm that keeps track of which request is being processed by which processing unit.
- This component is usually implemented using a typical web server with load-balancing capabilities (such as HA Proxy and Nginx).

5.9. Space-Based Architecture Style



Data grid



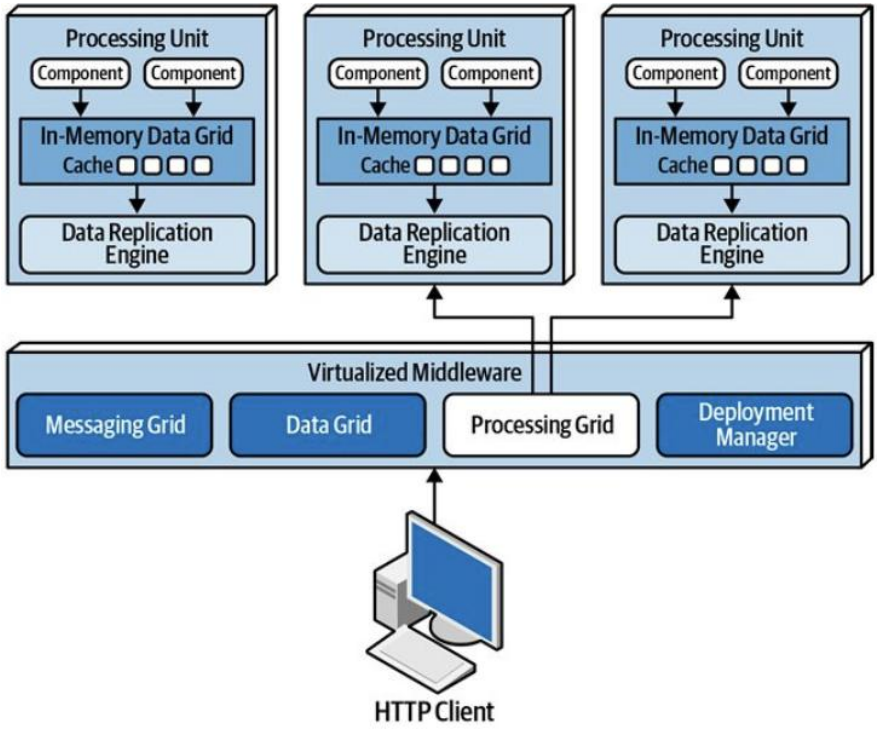
- *"The most important and crucial component in this architecture style."*
- In most modern implementations the data grid is **implemented solely within the processing units as a replicated cache**. However, for those replicated caching implementations that require an external controller, or when using a distributed cache, this functionality would reside in both the processing units as well as in the data grid component within the virtualized middleware.

- Data is synchronized between processing units that contain the same named data grid.

5.9. Space-Based Architecture Style



Processing grid



- Manages **input request** and **session state**
- When a request comes into the virtualized middleware, the messaging grid component **determines which active processing components are available to receive the request and forwards the request to one of those processing units.**
- This component is usually implemented using a typical web server with load-balancing capabilities (such as HA Proxy and Nginx).

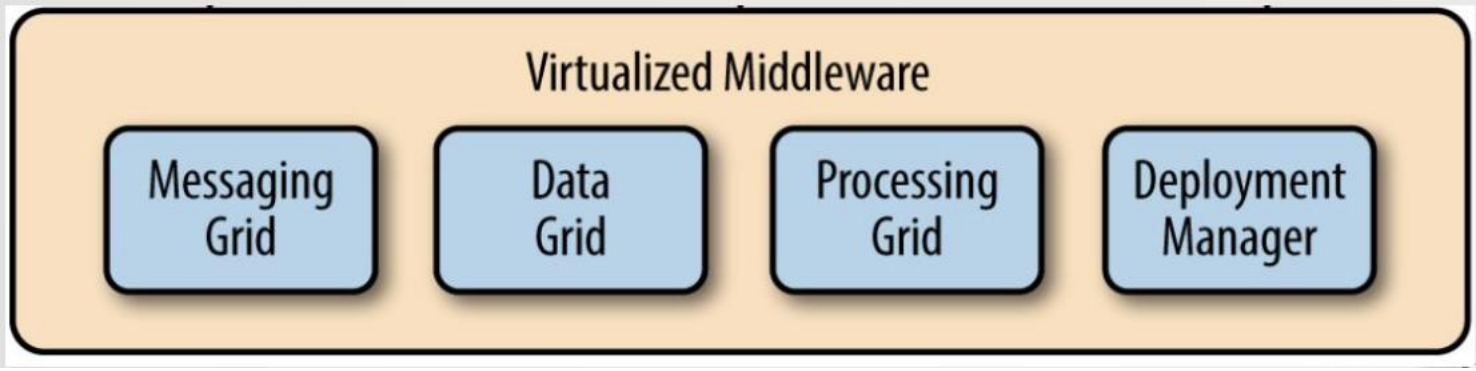
- The complexity can be varied from simple RR algorithm to a more complex next available e algorithm that keeps track of which request is being processed by which processing unit

5.9. Space-Based Architecture Style



Deployment Manager

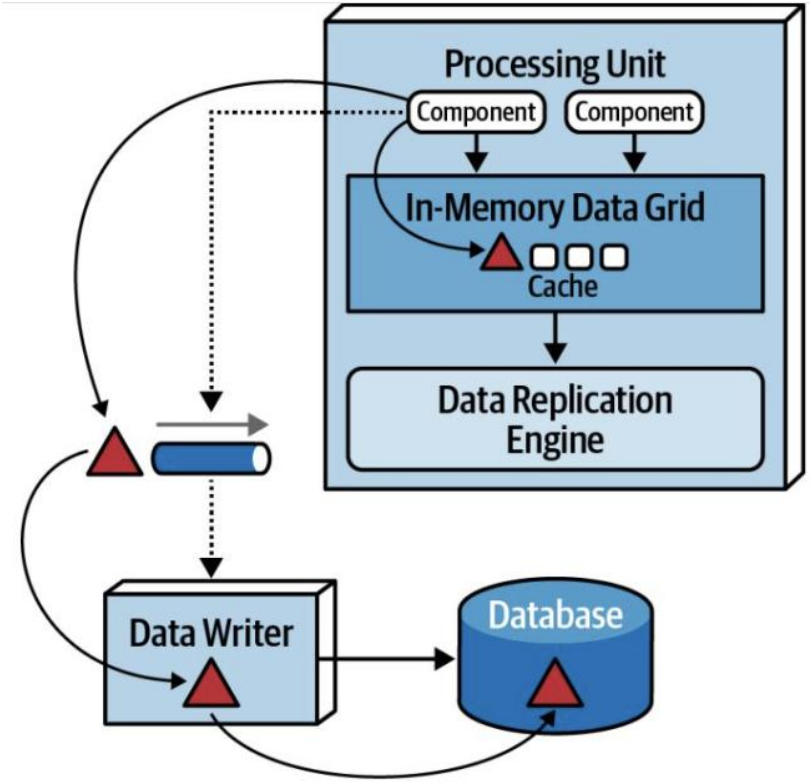
- The deployment manager component **manages the dynamic startup and shutdown of processing unit instances based on load conditions.**
- This component continually monitors response times and user loads, starts up new processing units when load increases, and shuts down processing units when the load decreases.
- It is a critical component to achieving variable scalability (elasticity) needs within an application.



5.9. Space-Based Architecture Style



Data Pumps

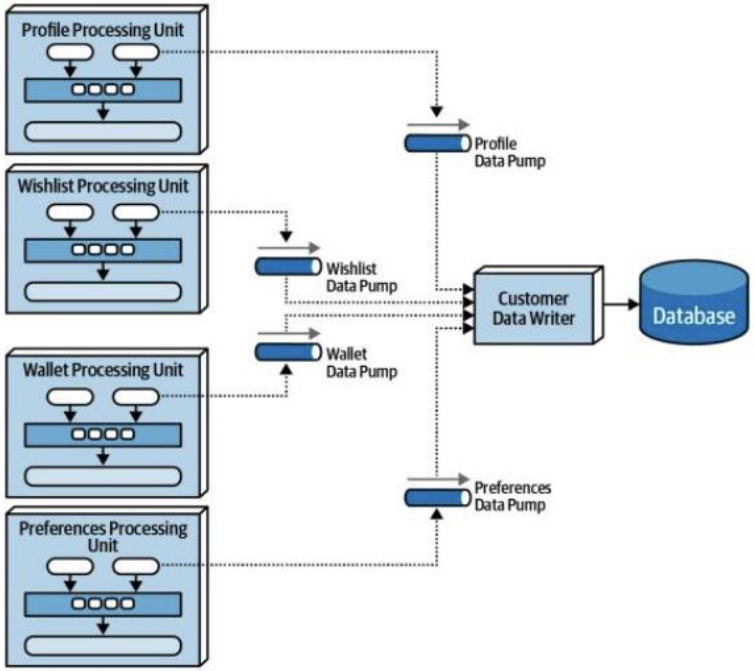


- A data pump is a **way of sending data** to another processor which then updates data in a database. (processing units do not directly read from and write to a database)
- Data pumps within a space-based architecture are always **asynchronous, providing eventual consistency** with the in-memory cache and the database.
- In most cases there are multiple data pumps, each one usually dedicated to a particular domain or subdomain (such as customer or inventory)

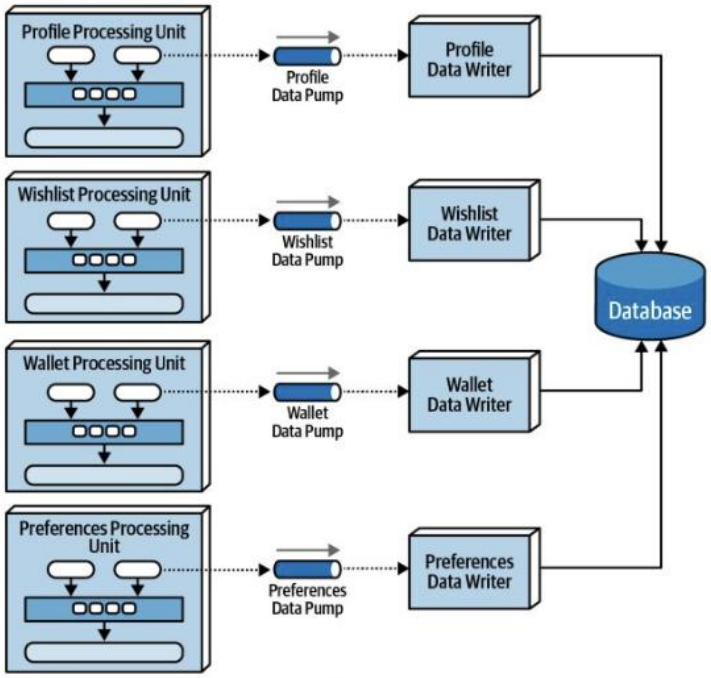
- Data pumps usually have **associated contracts**, including an action associated with the contract data (add, delete, or update). The contract can be JSON schema, XML schema, object or value-driven message.

5.9. Space-Based Architecture Style

Data Writers



Domain-based data writers



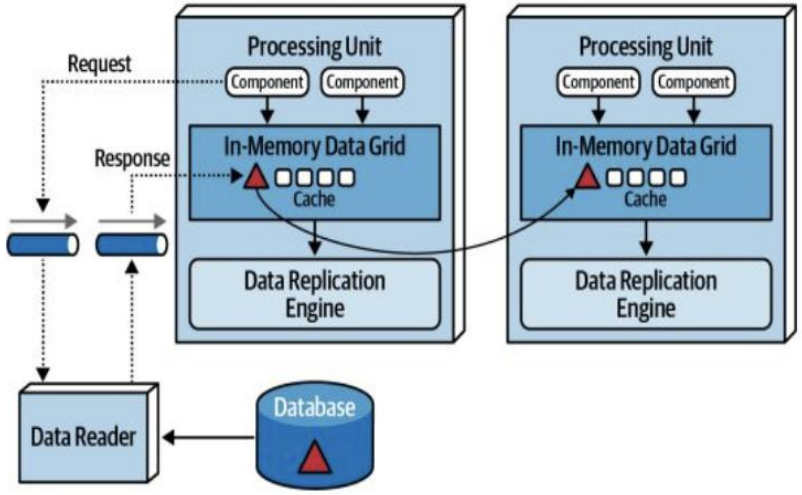
Dedicated data writers for each data pumps

- The **data writer component accepts messages from a data pump and updates the database** with the information contained in the message of the data pump.
- Data writers can be implemented as services, applications, or data hubs (such as Ab Initio)
- Each class of processing unit can have its own data writers or use same data writers with the others.

5.9. Space-Based Architecture Style



Data Reader



Data reader with reverse data pump

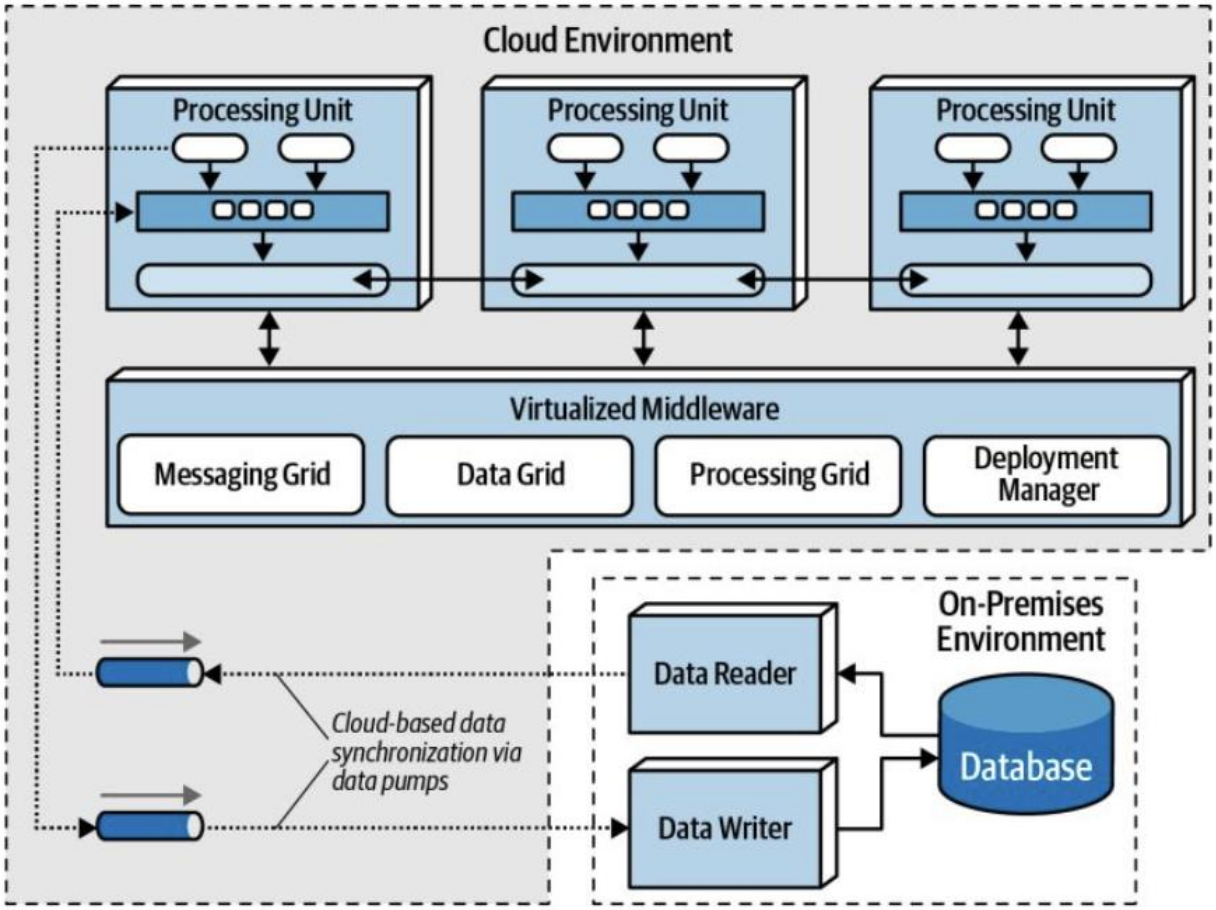
- Data readers take on the responsibility for **reading data from the database and sending it to the processing units** via a reverse data pump.
- In space-based architecture, data readers are only invoked under one of three situations:
  - a crash of all processing unit instances of the same named cache,
  - a redeployment of all processing
  - units within the same named cache, or retrieving archive data not contained in the replicated cache.

- Like data writers, data readers can also be domain-based or dedicated to a specific class of processing unit (which is usually the case)
- The **data writers and data readers** essentially form what is usually known as a **data abstraction layer** (or data access layer in some cases).

5.9. Space-Based Architecture Style



Cloud vs On-premises



Hybrid cloud-based and on-prem topology

5.9. Space-Based Architecture Style



Data Collision

- When using replicated caching in an active/active state where updates can occur to any service instance containing the same named cache, there is the possibility of a data collision due to replication latency.
  - A data collision occurs when data is updated in one cache instance (cache A), and during replication to another cache instance (cache B), the same data is updated by that cache (cache B).
  - In this scenario**, the local update to cache B will be overridden through replication by the old data from cache A**, and through replication the same data in cache A will be overridden by the update from cache B.

5.9. Space-Based Architecture Style



Data Collision

- There are several factors that influence how many data collisions might occur:
  - The number of processing unit instances containing the same cache
  - The update rate of the cache
  - The cache size
  - The replication latency of the caching product

5.9. Space-Based Architecture Style

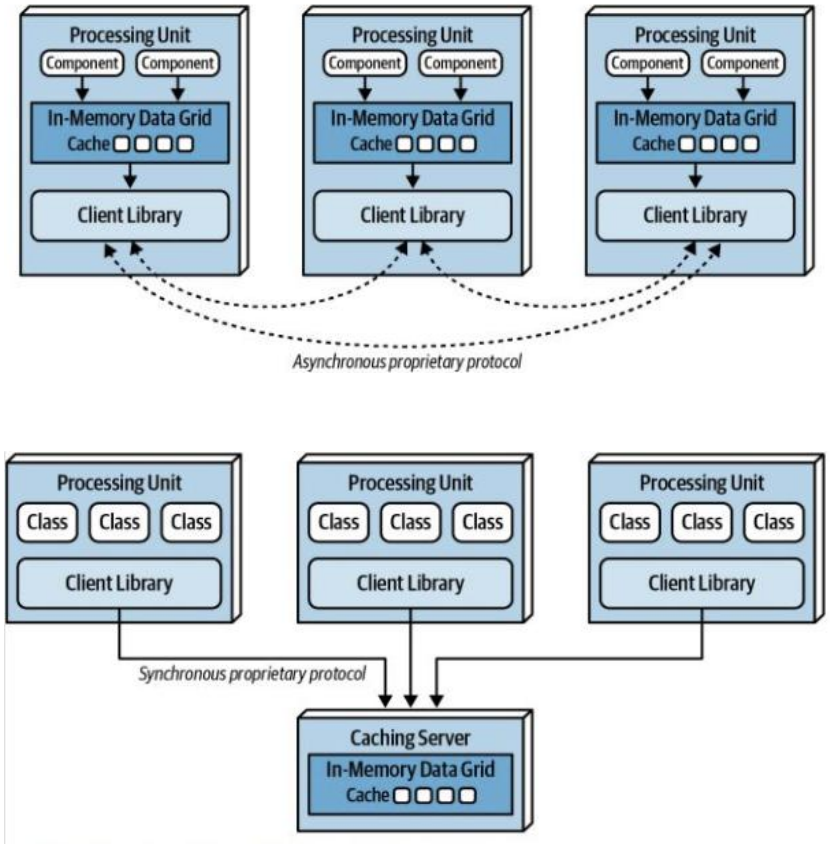


Data Collision

- **CollisionRate=N*UR2S*RL**
- Where
  - N represents the number of service instances using the same named cache,
  - UR represents the update rate in milliseconds (squared),
  - S the cache size (in terms of number of rows), and
  - RL the replication latency of the caching product.)

5.9. Space-Based Architecture Style

Replicated vs Distributed Caching



Decision criteria	Replicated cache	Distributed cache
Optimization	Performance	Consistency
Cache size	Small (<100 MB)	Large (>500 MB)
Type of data	Relatively static	Highly dynamic
Update frequency	Relatively low	High update rate
Fault tolerance	High	Low

Table 15-1. Distributed versus replicated caching

Distributed caching between processing units

5.9. Space-Based Architecture Style



Application

- Space-based architecture is well suited for applications that experience high spikes in user or request volume and applications that have throughput in excess of 10,000 concurrent users.
- Examples of space-based architecture include applications like online concert ticketing systems and online auction systems. Both of these examples require high performance, high scalability, and high levels of elasticity

5.9. Space-Based Architecture Style



Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1 to many
Deployability	★★★
Elasticity	★★★★★
Evolutionary	★★★
Fault tolerance	★★★
Modularity	★★★
Overall cost	★★
Performance	★★★★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★



Q&A